==========

FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION

==========

1. INTRODUCTION

Project Title : IMovies - Movie Ticket Booking

Team Member : Prachi Pandey - Full Stack Developer (Frontend, Backend, Database, Deployment)

2. PROJECT OVERVIEW

Purpose:

IMovies is a full stack web application that lets users browse movies, check show details, select seats and book tickets online. The goal is to replace manual ticket counters with a simple, fast and secure online booking experience.

Key Features:

- User registration and login with secure authentication (JWT)
- Browse the list of available movies with details
- Select show, date, time and seats
- Book tickets and view booking confirmation
- View booking history
- Responsive design that works on mobile and desktop
- Admin-side management of movies and shows [EDIT: remove if not implemented]

3. ARCHITECTURE

Frontend (React.js):

- Built as a single page application using React.js
- Component-based structure with React Router for navigation
- Axios / Fetch used for async API calls to the backend
- Styled with HTML, CSS and Bootstrap for a responsive layout

Backend (Node.js and Express.js):

- RESTful API built with Node.js and Express.js
- Routes handle users, movies and bookings
- Middleware handles JWT verification, CORS and error handling
- Async/await used for all database and API operations

Database (MongoDB):

- MongoDB with Mongoose for schema modelling
- Main collections:
  Users : name, email, password (hashed), role
  Movies : title, description, genre, duration, language, poster, showtimes
  Bookings : user, movie, show date and time, seats, total price, booking date
- Bookings reference Users and Movies using ObjectId references

4. SETUP INSTRUCTIONS

Prerequisites:

- Node.js (v16 or higher) and npm
- MongoDB (local installation or MongoDB Atlas account)
- Git

Installation:

1. Clone the repository

git clone https://github.com/prachi-pandey-13/IMovies---Movie-Ticket-Booking.git

2. Move into the project folder

cd IMovies---Movie-Ticket-Booking

3. Install backend dependencies

cd server

npm install

4. Install frontend dependencies

cd ../client

npm install

5. Create a .env file inside the server folder with:

PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

5. FOLDER STRUCTURE

Client (React frontend):

client/

public/ - static files and index.html

src/

components/ - reusable UI components (Navbar, MovieCard, SeatSelector)

pages/ - page-level components (Home, Login, Register, Booking)

App.js - routes and main layout

index.js - entry point

Server (Node.js backend):

server/

models/ - Mongoose schemas (User, Movie, Booking)

routes/ - API route definitions

controllers/ - request handling logic

middleware/ - authentication and error handling

server.js - app entry point and database connection

.env - environment variables (not pushed to GitHub)

6. RUNNING THE APPLICATION

Backend:

cd server

npm start

Frontend:

cd client

npm start

The frontend runs on http://localhost:3000 and the backend on http://localhost:5000 by default.

7. API DOCUMENTATION

Authentication

POST /api/auth/register

Body : { "name": "...", "email": "...", "password": "..." }

Response : { "message": "User registered successfully" }

POST /api/auth/login

Body : { "email": "...", "password": "..." }

Response : { "token": "<jwt_token>", "user": { ... } }

Movies

GET /api/movies

Response : [{ "_id": "...", "title": "...", "genre": "...", ... }]

GET /api/movies/:id

Response : { "_id": "...", "title": "...", "description": "...", ... }

Bookings (requires JWT token)

POST /api/bookings

Body : { "movieId": "...", "showTime": "...", "seats": ["A1", "A2"] }

Response : { "message": "Booking confirmed", "booking": { ... } }

GET /api/bookings/my

Response : [{ "movie": "...", "seats": [...], "date": "..." }]

8. AUTHENTICATION

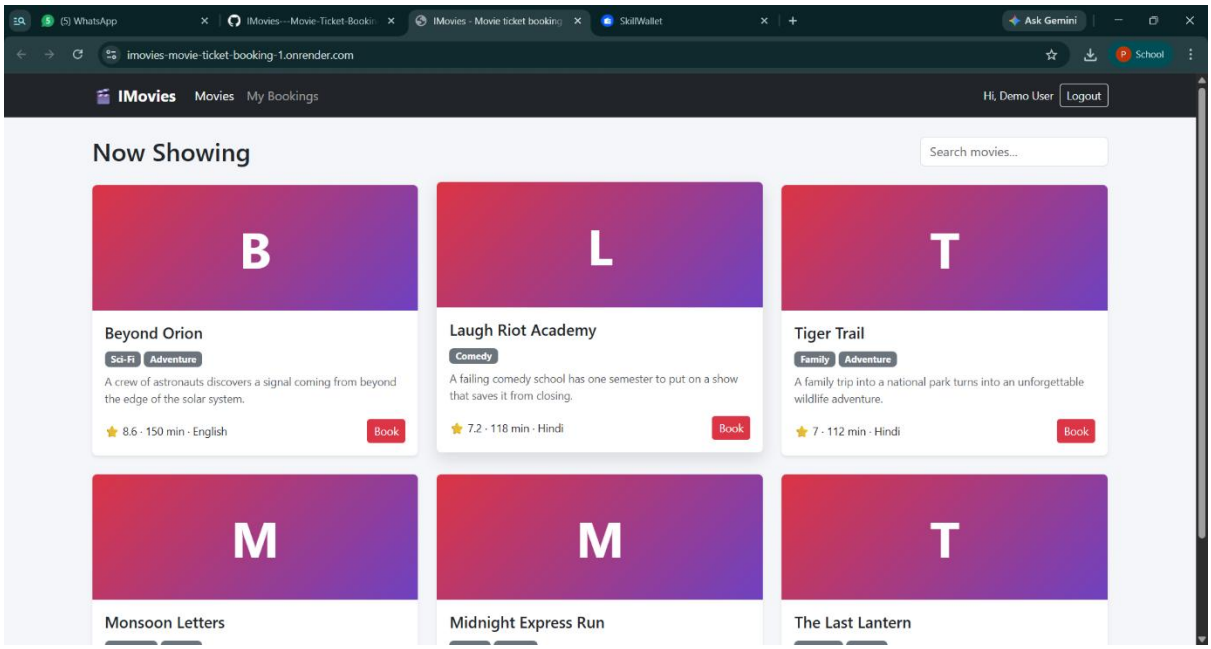
- Passwords are hashed with bcrypt before being stored in MongoDB.
- On login, the server issues a JSON Web Token (JWT) signed with a secret key.
- The client stores the token and sends it in the Authorization header:

  Authorization: Bearer <token>
- A middleware on the server verifies the token before allowing access to

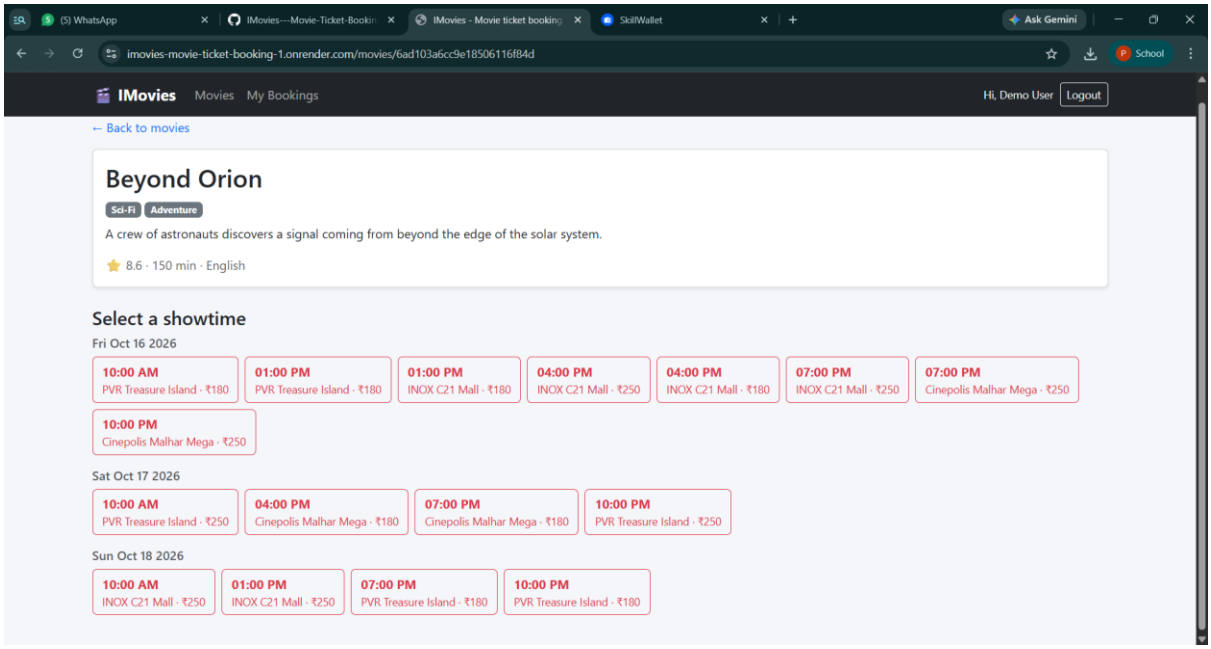
  protected routes such as booking and viewing booking history.
- Role-based access separates normal users from admins.

9. USER INTERFACE

- Home Page : screenshots/home.png

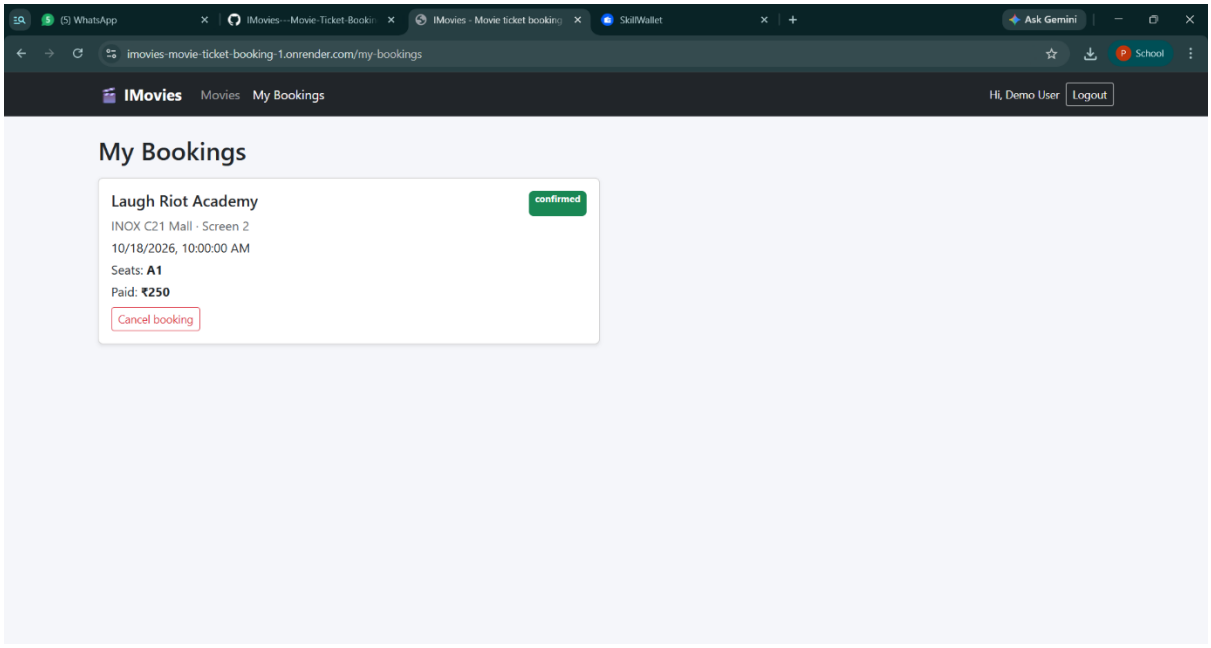


- Movie Details : screenshots/movie-details.png



- Seat Selection : screenshots/seat-selection.png

- Booking Confirmation: screenshots/confirmation.png



- Login / Register : screenshots/login.png

10. TESTING

- Manual testing of all user flows: register, login, browse, book, view bookings
- API endpoints tested using Postman
- Browser DevTools used to debug frontend and network requests
- Tested on multiple screen sizes for responsiveness

11. SCREENSHOTS OR DEMO

Live Demo : https://imovies-movie-ticket-booking-1.onrender.com

GitHub Repo : https://github.com/prachi-pandey-13/IMovies---Movie-Ticket-Booking

Note: The app is hosted on Render's free tier, so the first load may take

30-60 seconds while the server wakes up.

12. KNOWN ISSUES

- First load on the deployed app can be slow due to free-tier hosting.
- No real payment gateway; payment is simulated. [EDIT: adjust if different]
- [EDIT: add any other bugs you know of]

13. FUTURE ENHANCEMENTS

- Integrate a real payment gateway (Razorpay / Stripe)
- Email or SMS ticket confirmation with QR code
- Real-time seat locking to prevent double booking
- Movie reviews and ratings
- Cinema and city selection
- Cancellation and refund option
- Admin analytics dashboard